

Универсални функции, които
приемат поведение като
параметър

1

Рекурсивен Map (с Func)

```
static List<TOut> Map<TIn, TOut>(List<TIn> list, Func<TIn, TOut> f)
{
    if (list.Count == 0)
        return new List<TOut>();

    return new List<TOut> { f(list[0]) }
        .Concat(Map(list.Skip(1).ToList(), f))
        .ToList();
}
```

1

Примери за приложение на Map

- ◆ Доблиране на елементите на списък

```
var nums = new List<int> { 1, 2, 3, 4 };  
var doubled = Map(nums, x => x * 2);
```

- ◆ Повдигане на квадрат

```
var squared = Map(nums, x => x * x);
```

- ◆ Преобразуване от Целзий във Фаренхайт

```
var temps = new List<double> { 0, 10, 20 };  
var fahrenheit = Map(temps, c => c * 9 / 5 + 32);
```

1

Примери за приложение на Map

◆ Абсолютна стойност

```
var nums = new List<int> { -5, 3, -2 };  
var absValues = Map(nums, x => Math.Abs(x));
```

◆ Дължина на всеки string

```
var words = new List<string> { "cat", "elephant", "dog" };  
var lengths = Map(words, w => w.Length);
```

◆ Преобразуване в главни букви

```
var words = new List<string> { "hello", "world" };  
var upper = Map(words, w => w.ToUpper());
```

1

Примери за приложение на Map

◆ Добавяне на префикс

```
var names = new List<string> { "Ivan", "Maria" };  
var titled = Map(names, n => "Student: " + n);
```

◆ Промяна на тип (int → string)

```
var nums = new List<int> { 1, 2, 3 };  
var asText = Map(nums, x => "Number: " + x);
```

◆ Комбиниране на операции

```
var nums = new List<int> { 1, 2, 3, 4 };  
var result = Map( nums, x => (x * 3).ToString());
```

1

Примери за приложение на Map

♦ Работа с обекти

```
class Student
{
    public string Name { get; set; }
    public double Grade { get; set; }
}
```

♦ Извличане само на имената

```
var students = new List<Student>
{
    new Student { Name = "Ivan", Grade = 5.50 },
    new Student { Name = "Maria", Grade = 6.00 }
};

var names = Map(students, s => s.Name);
```

1

Примери за приложение на Map



Изчисляване на повишена оценка

```
var updatedGrades = Map(students, s => s.Grade + 0.5);
```

Мар винаги:

- не променя оригиналния списък
- връща нов списък
- запазва броя на елементите
- прилага една и съща логика върху всеки елемент
- позволява смяна на тип

2

Рекурсивен Filter (с Predicate)

```
static List<T> Filter<T>(List<T> list, Predicate<T> predicate)
{
    if (list.Count == 0)
        return new List<T>();
    var head = list[0];
    var tail = Filter(list.Skip(1).ToList(), predicate);
    if (predicate(head))
        return new List<T> { head }.Concat(tail).ToList();
    return tail;
}
```

Примери за приложение на Filter

- ◆ Филтриране на отрицателни числа

```
var nums = new List<int> { -5, 3, -2, 7 };
```

```
var negatives = Filter(nums, x => x < 0);
```

- ◆ Филтриране на числа в интервал

```
var nums = new List<int> { 5, 10, 15, 20 };
```

```
var between10And20 = Filter(nums, x => x >= 10 && x <= 20);
```

- ◆ Филтриране на четни и по-големи от 10

```
var nums = new List<int> { 2, 11, 14, 7, 20 };
```

```
var result = Filter(nums, x => x % 2 == 0 && x > 10);
```

Примери за приложение на Filter

- ◆ Филтриране на string по дължина

```
var words = new List<string> { "cat", "elephant", "dog", "house" };  
var longWords = Filter(words, w => w.Length > 4);
```

- ◆ Филтриране на string, започващи с главна буква

```
var names = new List<string> { "Ivan", "maria", "Petar", "georgi" };  
var capitalized = Filter(names, n => Char.IsUpper(n[0]));
```

- ◆ Премахване на null стойности

```
var values = new List<string> { "hello", null, "world", null };  
var withoutNulls = Filter(values, x => x != null);
```

Примери за приложение на Filter



Filter + логическа композиция

```
Predicate<int> isEven = x => x % 2 == 0;
```

```
Predicate<int> greaterThanTen = x => x > 10;
```

```
var nums = new List<int> { 4, 8, 12, 15, 20 };
```

```
var result = Filter(nums, x => isEven(x) && greaterThanTen(x));
```



Filter + външна колекция

```
var allowed = new List<int> { 2, 4, 6 };
```

```
var nums = new List<int> { 1, 2, 3, 4, 5, 6 };
```

```
var result = Filter(nums, x => allowed.Contains(x));
```

Filter винаги:

- намалява броя на елементите
- запазва типа
- не променя оригиналния списък
- връща нов списък
- логиката идва от Predicate

3

Рекурсивен Fold (Reduce) с Func

```
static TAcc Fold<T, TAcc>(List<T> list, TAcc acc, Func<TAcc, T, TAcc> f)
{
    if (list.Count == 0)
        return acc;

    return Fold( list.Skip(1).ToList(), f(acc, list[0]), f );
}
```

Примери за приложение на Fold



Намиране на максимум

```
var nums = new List<int> { 5, 2, 10, 3 };
```

```
var max = Fold(nums, nums[0], (acc, x) => x > acc ? x : acc);
```



Броење на елементи

```
var count = Fold(nums, 0, (acc, x) => acc + 1);
```



Сумиране само на четни числа

```
var evenSum = Fold(nums, 0, (acc, x) => x % 2 == 0 ? acc + x : acc);
```

Примери за приложение на Fold



Конкатенация в string

```
var nums = new List<int> { 1, 2, 3, 4 };  
var text = Fold(nums, "", (acc, x) => acc + x.ToString());
```



Създаване на списък (имплементация на Map чрез Fold)

```
var doubled = Fold(nums, new List<int>(),  
    (acc, x) => acc.Concat(new List<int> { x * 2 }).ToList());
```



Имплементация на Filter чрез Fold

```
var evens = Fold(nums, new List<int>(), (acc, x) =>  
    x % 2 == 0? acc.Concat(new List<int> { x }).ToList(): acc);
```


Примери за приложение на Fold



Проверка дали всички числа са положителни (All)

```
var allPositive = Fold(nums, true, (acc, x) => acc && x > 0);
```



Проверка дали съществува четно число (Any)

```
var hasEven = Fold(nums, false, (acc, x) => acc || x % 2 == 0);
```



Намиране на най-дългата дума

```
var words = new List<string> { "cat", "elephant", "dog" };
```

```
var longest = Fold(words, "", (acc, x) => x.Length > acc.Length ? x : acc);
```

Примери за приложение на Fold



Обръщане на списък чрез Fold

```
var reversed = Fold(nums, new List<int>(),  
                    (acc, x) => new List<int> { x }.Concat(acc).ToList());
```



Създаване на CSV низ (Comma-Separated Values)

```
var nums = new List<int> { 10, 20, 30 };  
var csv = Fold(nums, "", (acc, x) => acc == "" ? x.ToString() : acc + "," + x);
```

```
Console.WriteLine(csv);
```

Резултат: 10,20,30

Fold е най-мощната абстракция:

- може да имплементира Sum
- може да имплементира Map
- може да имплементира Filter
- може да имплементира Any / All
- може да създава нови структури

Във функционалното програмиране:
Map и Filter могат да се реализират чрез Fold
но Fold не може да се реализира чрез Map или Filter.